Apellidos y Nombre: .....Grupo:.....DNI: .....

Examen Final, parte 1 (Temas 1 al 8)

- Duración de esta parte: 2 horas. Escribid las respuestas en el enunciado y entregad todas sus hojas con apellidos y nombre en la cabecera.
- No podéis utilizar calculadora, móvil, apuntes, etc. En hoja aparte se os da una "chuleta" con información útil para realizar los ejercicios.
- La solución se publicará el 17 de enero de 2024.

Pregunta 1 (1 punto)

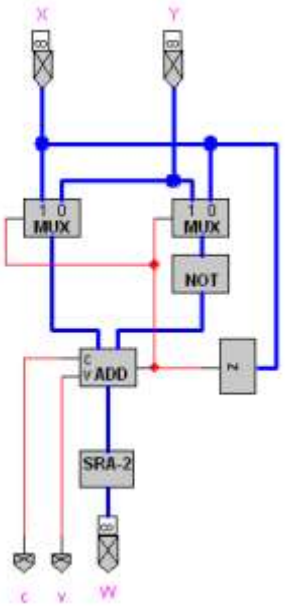
Cada fila de la tabla tiene 3 columnas con el vector X de 8 bits, X expresado en hexadecimal y el valor en decimal, X_u, que representa X interpretado como un número natural codificado en binario. Completa las casillas vacias.

Criterio de correción: binario, 0,25 cada casilla

X	X (hexa)	X_u
11000100	C4	196
01111000	78	120

Pregunta 2 (1,5 puntos)

Dado el siguiente circuito combinacional, los valores de los vectores de 8 bits de entrada X e Y, rellenad la tabla indicando el valor de las señales de salida c, v, el valor del vector de 8 bits de la salida W para cada combinación de valores de entrada (cada fila de la tabla). También tenéis que escribir el valor que representa W interpretado como un número natural codificado en binario (W_u) y el valor que representa W interpretado como un número entero codificado en complemento a dos (W_s).

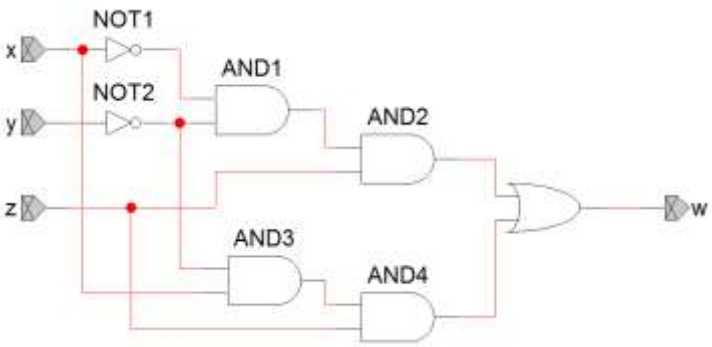


Criterio de correción: binario, 0,5 cada fila

X	Y	c	v	W	W_u	W_s
10001000	00010001	0	1	1110 0010	226	-30
11111111	00011111	0	0	0000 0111	7	7
00000000	10000000	0	1	1110 0000	224	-32

Pregunta 3 (2 puntos)

A partir del siguiente esquema:



a) Completad la tabla de verdad para la salida **W**

x	y	z	W
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

b) Obtener la expresión en suma de minterms de la salida **W**:

$m_1(x, y, z) + m_5(x, y, z) = !x \cdot !y \cdot z + x \cdot !y \cdot z$

c) Escribid el camino crítico (todos si hay varios) para el esquema del circuito anterior y su tiempo de propagación. Considerar los siguientes tiempos de propagación de las puertas lógicas: $Tp_{(Not)} = 10$, $Tp_{(And-2)} = 20$, $Tp_{(Or-2)} = 20$.

Camino crítico:	Tp = 70 ut
X -> NOT1 -> AND1 -> AND2 -> Or -> W Y -> NOT2 -> AND1 -> AND2 -> Or -> W Y -> NOT2 -> AND3 -> AND4 -> Or -> W	

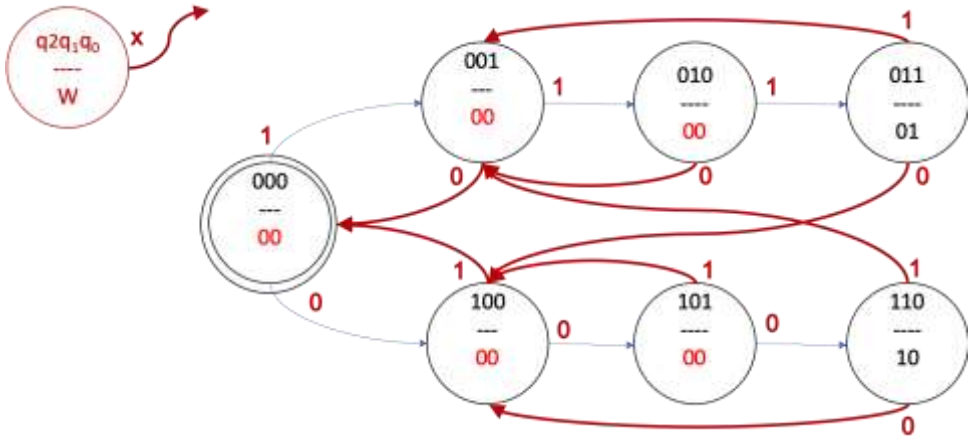
Criterio de corrección:

a) La tabla de verdad vale 1 punto: -0,5 puntos por cada bit incorrecto.
b) +0,25 puntos si la expresión es correcta y la tabla también estaba correcta.
c) +0,5 puntos si los caminos críticos son correctos. 0 puntos si hay algún camino NO crítico
d) +0,25 si el tiempo de ciclo es correcto (sólo si los caminos críticos son correctos)

Pregunta 4 (2 puntos)

Se desea implementar implementar un circuito que detecte si el número entre 0's y 1's de la entrada (x) se desequilibra demasiado. Se considera desequilibrado si llegan 3 ceros o más que unos o 3 unos o mas que ceros. Una vez detectado el desequilibrio se dará una señal de alarma durante un ciclo codificada de la siguiente forma: w=10 si hay más ceros que unos y W=01 si hay mas unos que ceros. Una vez dada la alarma se volverá a iniciar la comprobación del equilibrio teniendo ya en cuenta la entrada (x) en el ciclo que hemos dado la señal de alarma. Mientras no demos alarma, el valor de W ha de ser 00. Por ejemplo, la secuencia 010111011 tiene 3 ceros y 6 unos por lo que al llegar el último uno debería activar la alarma de demasiados unos y volver a iniciar la comprobación a partir de siguiente valor de x, el que ha llegado en ciclo en que se ha activado la alarma. Completad el grafo del circuito (incluir la leyenda).

Criterio de corrección: 0 si no hay leyenda o es incorrecta. -0,2 por cada estado incorrecto ya sea por las salidas o por las transiciones que de él salen.

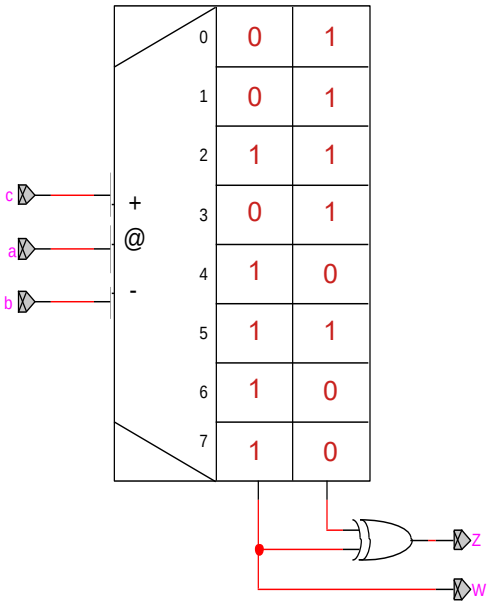


Pregunta 5 (1 punto)

Escribe el contenido de la ROM para que haga la función de un circuito cuyo funcionamiento está descrito en la siguiente tabla de verdad.

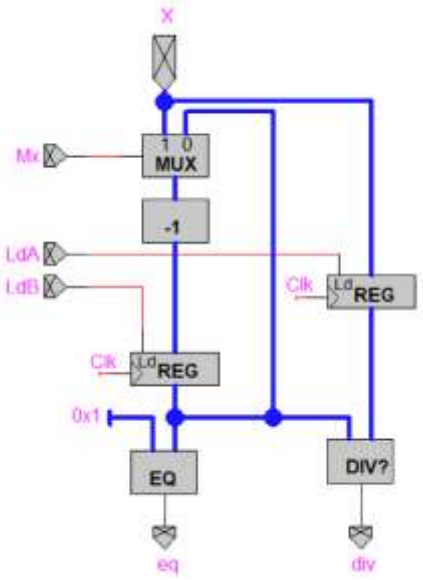
Criterio de corrección: binaria, 0,5 por cada columna.

a	b	c	Z	W
0	0	0	1	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	1



Pregunta 6 (2,5 puntos)

Queremos implementar un Procesador de Propósito Específico (PPE) que calcula si un número es primo o no (número divisible sólo por sí mismo y por 1). Para ello disponemos de la Unidad de Proceso de 16 bits de la figura. En dicha figura, el componente "DIV" tiene como entrada dos números de 16 bits y retorna si son divisibles entre sí (1 si lo son, 0 si no lo son).

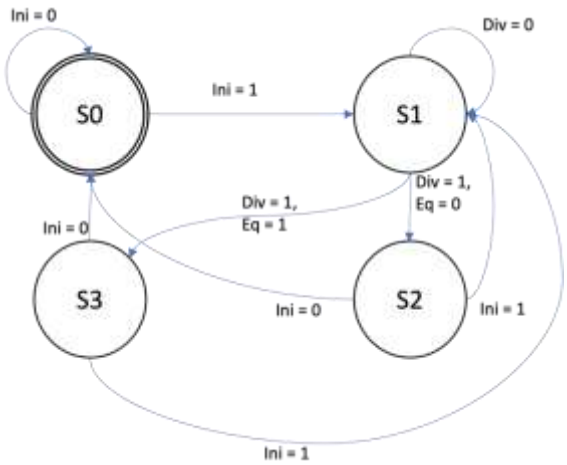


El circuito funciona de la siguiente forma. Tiene dos entradas: un bus de entrada de 16 bits y una señal de control ini. También tiene dos señales de salida: fi y primo. Cunado ini toma valor 1, en X tenemos el número que queremos comprobar si es o no primo. Pasados los ciclos que sean necesarios, la señal fi valdrá 1 y primo valdrá 1 si el número es primo y 0 si no lo es. Mientras fi valga 0, el valor de primo no importa.
Ignoraremos la señal de ini durante los cálculos e intentaremos poder empezar un nuevo cálculo lo antes posible. El valor recibido per el bus X siempre será mayor que 1.

1) Especifica las entradas (0,2 puntos) (corrección binaria)

   Ini, div i Eq

2) Diseña la Unidad de Control completando el siguiente grafo (1 punto) (-0,25 por cada estado con algún error)



	Mx	LdA	LdB	fi	primo
S0	1	1	1	0	x
S1	0	0	1	0	x
S2	1	1	1	1	0
S3	1	1	1	1	1

3) Si implementamos la unidad de control con 1 ROM y 1 Multiplexor de buses (0,5 puntos) (-0,2 por cada respuesta errónea)

   a) ¿Cuántas palabras tendrá la ROM? 2^2 = 4
   b) ¿Cuántos bit por palabra? 2*2^3+5 = 21
   c) ¿Cuántos bits tendrá el registro de estado? 2
   d) ¿Cuántos bits tendrá el selector del multiplexor de buses? 3

4) Si quisiéramos implementar el CLC de salida con un decodificador y puertas OR-2 (0,3 puntos) (-0,2 por cada respuesta errónea)

   a) ¿Cuántas entradas tendría el decodificador? 2
   b) ¿Cuántas OR-2 necesitaríamos para implementar la salida Mx? 2

5) Si quisiéramos implementar el CLC de estado siguiente con una ROM con las entradas en el siguiente orden (de mayor a menos peso): q1, q0, ini, eq, fi, ¿qué valor tendrían las siguientes palabras de la ROM? (0,5 puntos) (-0,1 por cada respuesta errónea)

   a) ROM[0x00] = 0x0
   b) ROM[0x1F] = 0x1
   c) ROM[0x08] = 0x1
   d) ROM[0x17] = 0x1
   e) ROM[0x0F] = 0x3